

•

题目 ID	标题	通过率	通过人数	尝试人数	添加时间
250401	白细胞计数	100%	7	7	2025-05
250402	整数奇偶排序	100%	6	6	2025-05
250403	和为给定数	100%	9	9	2025-05
250405	Freda 的越野跑	100%	6	6	2025-05
250404	月度开销	100%	11	11	2025-05

• »[2025 年数算 B13 班模拟机考\(复习\)](#) (7 题)

最新题目

题目 ID	标题	通过率	通过人数	尝试人数	添加时间
25TF7	木材加工	97%	33	34	2025-0
25TF6	最大子矩阵	90%	26	29	2025-0
25TF4	多余的括号（数据增强）	48%	11	23	2025-0
25TF5	康托集的图像表示	100%	30	30	2025-0
25TF3	牛的选举	97%	33	34	2025-0

• »[\(2025\) 作业六：图及算法](#)(8 题)

最新题目

题目 ID	标题	通过率	通过人数	尝试人数	添加时间
250608	兔子与星空	100%	65	65	2025-05
250607	兔子与樱花	98%	64	65	2025-05
250606	Subway	98%	62	63	2025-05
250605	Sorting It All Out	97%	68	70	2025-05
250604	拯救行动	99%	72	73	2025-05

• »[排序与查找小测复习](#)(3 题)

最新题目

题目 ID	标题	通过率	通过人数	尝试人数	添加时间
25T33	蚂蚁王国的越野跑	100%	12	12	2025-04-10 14:00
25T32	厚道的调分方法	94%	16	17	2025-04-10 14:00
25T31	充实的寒假生活	100%	14	14	2025-04-10 14:00

• [» \(2025\) 作业五：树及算法\(8 题\)](#)

最新题目

题目 ID	标题	通过率	通过人数	尝试人数	添加时间
8	宝藏二叉树	99%	92	93	2025-03-26 14:00
7	简单的树上最近公共祖先查询	100%	93	93	2025-03-26 14:00
6	树的重量	90%	85	94	2025-03-26 14:00
5	合并果子	99%	96	97	2025-03-26 14:00
4	树的转换	95%	91	96	2025-03-26 14:00

• [» \(2025\) 作业四：查找与排序\(8 题\)](#)

最新题目

题目 ID	标题	通过率	通过人数	尝试人数	添加时间
250408	矩形分割	93%	88	95	2025-03-26 14:00
250407	求逆序对数	100%	96	96	2025-03-26 14:00
250406	两座孤岛最短距离	100%	94	94	2025-03-26 14:00
250405	Freda 的越野跑	98%	95	97	2025-03-26 14:00
250404	月度开销	100%	97	97	2025-03-26 14:00

• [» \(2025\) 作业三：递归与动态规划\(8 题\)](#)

最新题目

题目 ID	标题	通过率	通过人数	尝试人数	添加时间
250308	最长的公共子序列的长度	100%	99	99	2025-03-26 14:00
250307	回文字符串	100%	98	98	2025-03-26 14:00
250306	奇数拆分	99%	97	98	2025-03-26 14:00

题目 ID	标题	通过率	通过人数	尝试人数	添加时
250305	小木棍	99%	96	97	2025-
250304	求斐波那契数列第 k 项(递归)	100%	99	99	2025-

- [» \(2025\) 作业二：线性表\(16 题\)](#)

最新题目

题目 ID	标题	通过率	通过人数	尝试人数	添加时
8	位查询（数据加强版）	106%	18	17	2025-0
7	Queue	95%	18	19	2025-0
6	波兰表达式	106%	17	16	2025-0
5	放苹果（盘子相同）	100%	15	15	2025-0
4	多项式加法	127%	19	15	2025-0

- [» \(2025\) 作业一：Python 入门\(5 题\)](#)

最新题目

题目 ID	标题	通过率	通过人数	尝试人数	添加时
250105	螺旋加密	96%	100	104	2025-
250104	Maximum 23 Number	99%	102	103	2025-
250103	矩形切割	100%	103	103	2025-
250102	字符串中的第一个不重复字符	99%	102	103	2025-
250101	泰波拿契数	100%	105	105	

250401:白细胞计数

查看提交统计提问

总时间限制: 1000ms 内存限制: 65536kB

描述

医院采样了某临床病例治疗期间的白细胞数量样本 n 份，用于分析某种新抗生素对该

病例的治疗效果。为了降低分析误差，要先从这 n 份样本中去除一个数值最大的样本和一个数值最小的样本，然后将剩余 $n-2$ 个有效样本的平均值作为分析指标。同时，为了观察该抗生素的疗效是否稳定，还要给出该平均值的误差，即所有有效样本（即不包括已扣除的两个样本）与该平均值之差的绝对值的最大值。

现在请你编写程序，根据提供的 n 个样本值，计算出该病例的平均白细胞数量和对应的误差。

输入

输入的第一行是一个正整数 n ($2 < n \leq 300$)，表明共有 n 个样本。

以下共有 n 行，每行为一个浮点数，为对应的白细胞数量，其单位为 $10^9/L$ 。数与数之间以一个空格分开。

输出

输出为两个浮点数，中间以一个空格分开。分别为平均白细胞数量和对应的误差，单位也是 $10^9/L$ 。计算结果需保留到小数点后 2 位。

样例输入

5

12.0

13.0

11.0

9.0

10.0

样例输出

11.00 1.00

提示

为避免浮点精度误差过大，请使用 `double` 类型。

250402:整数奇偶排序

总时间限制:

1000ms

内存限制:

65536kB

描述

给定 10 个整数的序列，要求对其重新排序。排序要求：

- 1.奇数在前，偶数在后；
- 2.奇数按从大到小排序；
- 3.偶数按从小到大排序。

输入

输入一行，包含 10 个整数，彼此以一个空格分开，每个整数的范围是大于等于 0，小于等于 100。

输出

按照要求排序后输出一行，包含排序后的 10 个整数，数与数之间以一个空格分开。

样例输入

4 7 3 13 11 12 0 47 34 98

样例输出

47 13 11 7 3 0 4 12 34 98

250403:和为给定数

总时间限制:

1000ms

内存限制:

65536kB

描述

给出若干个整数，询问其中是否有一对数的和等于给定的数。

输入

共三行:

第一行是整数 n ($0 < n \leq 100,000$), 表示有 n 个整数。

第二行是 n 个整数。整数的范围是在 0 到 10^8 之间。

第三行是一个整数 m ($0 \leq m \leq 2^{30}$), 表示需要得到的和。

输出

若存在和为 m 的数对，输出两个整数，小的在前，大的在后，中间用单个空格隔开。

若有多个数对满足条件，选择数对中较小的数更小的。若找不到符合要求的数对，输出一行 No。

样例输入

4

2 5 1 4

6

样例输出

1 5

250405:Freda 的越野跑

描述

Freda 报名参加了学校的越野跑。越野跑共有 N 人参加，在一条笔直的道路上进行。这 N 个人在起点处站成一列，相邻两个人之间保持一定的间距。比赛开始后，这 N 个人同时沿着道路向相同的方向跑去。换句话说，这 N 个人可以看作 x 轴上的 N 个点，在比赛开始后，它们同时向 x 轴正方向移动。假设越野跑的距离足够远，这 N 个人的速度各不相同且保持匀速运动，那么会有多少对参赛者之间发生“赶超”的事件呢？

输入

第一行 1 个整数 N 。

第二行为 N 个非负整数，按从前到后的顺序给出每个人的跑步速度。

对于 50% 的数据， $2 \leq N \leq 1000$ 。

对于 100% 的数据， $2 \leq N \leq 100000$ 。

输出

一个整数，表示有多少对参赛者之间发生赶超事件。

样例输入

5

1 3 10 8 5

样例输出

7

提示

我们把这 5 个人依次编号为 A,B,C,D,E，速度分别为 1,3,10,8,5。

在跑步过程中：

B,C,D,E 均会超过 A，因为他们的速度都比 A 快；

C,D,E 都会超过 B，因为他们的速度都比 B 快；

C,D,E 之间不会发生赶超，因为速度快的起跑时就在前边。

250404:月度开销

描述

农夫约翰是一个精明的会计师。他意识到自己可能没有足够的钱来维持农场的运转了。他计算出并记录下了接下来 $N(1 \leq N \leq 100,000)$ 天里每天需要的开销。

约翰打算为连续的 $M(1 \leq M \leq N)$ 个财政周期创建预算案，他把一个财政周期命名为 fajo 月。每个 fajo 月包含一天或连续的多天，每天被恰好包含在一个 fajo 月里。

约翰的目标是合理安排每个 fajo 月包含的天数，使得开销最多的 fajo 月的开销尽可能少。

输入

第一行包含两个整数 N, M ，用单个空格隔开。

接下来 N 行，每行包含一个 1 到 10000 之间的整数，按顺序给出接下来 N 天里每天的开销。

输出

一个整数，即最大月度开销的最小值。

提示

若约翰将前两天作为一个月，第三、四两天作为一个月，最后三天每天作为一个月，则最大月度开销为 500。其他任何分配方案都会比这个值更大。

250406:两座孤岛最短距离

描述

给一个由 1 跟 0 组成的方形地图，1 代表土地，0 代表水域

相邻(上下左右 4 个方位当作相邻)的 1 组成孤岛

现在你可以将 0 转成 1，搭建出一个链接 2 个孤岛的桥

请问最少要将几个 0 转成 1，才能建成链接孤岛的桥。

题目中恰好有 2 个孤岛(顾答案不会是 0)

输入

一个正整数 n，代表几行输入

n 行 0 跟 1 字串

输出

一个正整数 k，代表最短距离

样例输入

3

110

000

001

样例输出

2

提示

样例输入中的两个孤岛最短距离为 2

250407:求逆序对数

描述

对于一个长度为 N 的整数序列 A ，满足 $i < j$ 且 $A_i > A_j$ 的数对 (i, j) 称为整数序列 A 的一个逆序

$i < j \leq n$ 且 $a_i > a_j$ 的数对 (i, j) 称为整数序列 A 的一个逆序

请求出整数序列 A 的所有逆序对个数

输入

输入包含多组测试数据，每组测试数据有两行

第一行为整数 N ($1 \leq N \leq 20000$)，当输入 0 时结束

第二行为 N 个整数，表示长为 N 的整数序列

输出

每组数据对应一行，输出逆序对的个数

样例输入

5

1 2 3 4 5

5

5 4 3 2 1

1

1

0

样例输出

0

10

0

250408:矩形分割

描述

平面上有一个大矩形，其左下角坐标 $(0, 0)$ ，右上角坐标 (R, R) 。大矩形内部包含一些小矩形，小矩形都平行于坐标轴且互不重叠。所有矩形的顶点都是整点。要求画一根平行于 y 轴的直线 $x=k$ (k 是整数)，使得这些小矩形落在直线左边的面积必须大于等于落在右边的面积，且两边面积之差最小。并且，要使得大矩形在直线左边的面积尽可能大。注意：若直线穿过一个小矩形，将会把它切成两个部分，分属左右两侧。

输入

第一行是整数 R ，表示大矩形的右上角坐标是 (R, R) ($1 \leq R \leq 1,000,000$)。

接下来的一行是整数 N ，表示一共有 N 个小矩形 ($0 < N \leq 10000$)。

再接下来有 N 行。每行有 4 个整数， L, T, W 和 H ，表示有一个小矩形的左上角坐标是 (L, T) ，宽度是 W ，高度是 H ($0 \leq L, T \leq R, 0 < W, H \leq R$)。小矩形不会有位于大矩形之外的部分。

输出

输出整数 n ，表示答案应该是直线 $x=n$ 。如果必要的话， $x=R$ 也可以是答案。

样例输入

1000

2

1 1 2 1

5 1 2 1

样例输出

5

250101:泰波拿契數

500ms

内存限制:

65536kB

描述

泰波拿契数列 T_n 定义是

$T_0 = 0, T_1 = 1, T_2 = 1$, and $T_{n+3} = T_n + T_{n+1} + T_{n+2}$ for $n \geq 0$.

给定 n 请算出 T_n

n 的范围: $1 \leq n \leq 30$

输入

一个正整数 n

输出

一个正整数 k

样例输入

4

样例输出

4

提示

$T_3 = 0 + 1 + 1 = 2$

$T_4 = 1 + 1 + 2 = 4$

250102:字符串中的第一个不重复字符

总时间限制:

1000ms

内存限制:

65536kB

描述

给定一个字符串 *s*，找到其中的第一个「只在 *s* 中出现过一次」的字符，并返回其对应的下标。 如果不存在这样的字符，则返回 -1。

输入

一个字符串，长度在 100,000 以内，只由小写英文字母组成

输出

第一个不重复字符对应的下标，或 -1

样例输入

perpendicular

样例输出

5

提示

如果输入是 "abcabc"，则没有任何一个字符是「只在 *s* 中出现过一次」的。此时返回 -1。

本题最好使用 Python 中的 dict 对象。

250103:矩形切割

总时间限制:

5000ms

单个测试点时间限制:

1000ms

内存限制:

65536kB

描述

小明有一些矩形的材料，他要从这些矩形材料中切割出一些正方形。

当他面对一块矩形材料时，他总是平行于较短的切割一刀，切出一块最大的正方形，剩下一块矩形，然后再切割剩下的矩形材料，直到全部切为正方形为止。

例如，对于一块两边长度分别为 5 和 3 的材料（记为 5×3 ），小明会依次切出 3×3 、 2×2 、 1×1 、 1×1 共 4 个正方形。

现在小明有一块矩形的材料，两边长分别是 N 和 M 。请问小明最终会切出多少个正方形？

输入

一行两个整数 N 和 M 。

$1 \leq N, M \leq 3000$

输出

一个整数代表答案

样例输入

5 3

样例输出

4

250104:Maximum 23 Number

总时间限制:

10000ms

内存限制:

65536kB

描述

给定一个只有 2,3 的正整数

能够将其中你最多一个数从 2 变成 3

请输出改变后最大的值

输入

一个正整数 n

$1 < n < 10^4$

输出

一个正整数

样例输入

2333

样例输出

3333

250105:螺旋加密

- [查看](#)
- [提交](#)
- [统计](#)
- [提问](#)

总时间限制:

1000ms

内存限制:

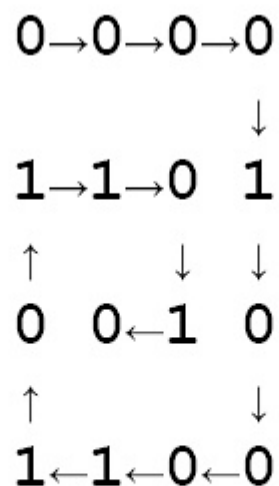
65536kB

描述

Chip 和 Dale 发明了一种文本信息加密技术。他们事先秘密约定好矩阵的行数和列数。接着，将字符按如下方式编码：

1. 所有文本只包含大写字母和空格。
2. 每个字符均赋予一个数值：空格=0，A=1，B=2，……，Y=25，Z=26。

按照下图所示的方式，将每个字符对应数值的 5 位二进制数依次填入矩阵。最后用 0 将矩阵补充完整。例如，对于信息“ACM”，行列数均为 4 时，矩阵将被填充为：



A = 00001, C = 00011, M = 01101
(one extra 0)

将矩阵中的数字按行连起来形成数字串，完成加密。例子中的信息最终会被加密为：0000110100101100。

输入

一行。首先是两个整数 $R(1 \leq R \leq 20)$ 和 $C(1 \leq C \leq 20)$ ，表示行数和列数。之后是一个只包含大写字母和空格的字符串。字符串的长度 $\leq (R \times C) / 5$ 。R 和 C 之间以及 C 和字符串之间均用单个空格隔开。

输出

一行，为加密后的二进制串。注意你可能需要用 0 将矩阵补充完整。

样例输入

4 4 ACM

样例输出

0000110100101100

250201:位查询

给出 N 个范围在 $[0, 65535]$ 的整数，编程支持以下的操作：

(1) 修改操作：C d ，所有的数都增加 d 。如果超过 65535，把结果模 65536。 $0 \leq d \leq 65535$

(2) 查询操作：Q i ，统计在 N 个正整数中有多少个整数其对应的二进制形式的第 i 位二进制位为非 0。 $0 \leq i \leq 15$ 。并且最低位 i 为 0。

最后，输出所有查询操作的统计值。

输入

输入的第一行为两个正整数 N, M ，其中 N 为操作的整数的个数，而 M 为具体有多少个操作。

输入的第二行为 N 个正整数，为进行操作的 N 个正整数。

下面有 M 行，分别表示 M 个操作。

$N \leq 100000, M \leq 200000$

输出

输出所有查询操作 Q 的统计值，每一个查询操作统计结果输出为一行。

样例输入

提示

只输出查询操作 Q 的统计值。

250202:神奇的幻方

描述

幻方是一个很神奇的 $N \times N$ 矩阵，它的每行、每列与对角线，加起来的数字和都是相同的。

我们可以通过以下方法构建一个幻方。（阶数为奇数）

1. 第一个数字写在第一行的中间

2. 下一个数字，都写在上一个数字的右上方：

a. 如果该数字在第一行，则下一个数字写在最后一行，列数为该数字的右一列

b. 如果该数字在最后一列，则下一个数字写在第一列，行数为该数字的上一行

c. 如果该数字在右上角，或者该数字的右上方已有数字，则下一个数字写在该数字的下方

输入

一个数字 N ($N \leq 20$)

输出

按上方法构造的 $2N-1 \times 2N-1$ 的幻方

样例输入

3

样例输出

17 24 1 8 15

23 5 7 14 16

4 6 13 20 22

10 12 19 21 3

11 18 25 2 9

250203:最后的最后

描述

[弗拉维奥·约瑟夫斯](#)是 1 世纪的一名犹太历史学家。他在自己的日记中写道，在一次战中，他和他的 40 个战友被罗马军队包围在洞中。他们讨论是自杀还是被俘，最终决定自杀，并以抽签的方式决定谁杀掉谁。约瑟夫斯和另外一个人是最后两个留下的人。约瑟夫斯说服了那个人，他们将向罗马军队投降，不再自杀。约瑟夫斯把他的存活归因于运气或天意，他不知道是哪一个。

在计算机科学与数学中，就有一个以此命名的问题：**约瑟夫斯问题**（有时也称为**约瑟夫斯置换**）。在计算机编程的算法中，类似问题又称为**约瑟夫环**。具体描述如下：有 n 个囚犯站成一个圆圈，准备处决。首先从一个人开始，越过 $k - 2$ 个人（因为第一个人已经被越过），并杀掉第 k 个人。接着，再越过 $k - 1$ 个人，并杀掉第 k 个人。这个过程沿着圆圈一直进行，直到最终只剩下一个人留下，这个人就可以继续活着。问题是，给定了 n 和 k ，一开始要站在什么地方才能避免被处决？

为了让大家熟悉循环链表的使用，对该题进行模拟。我们要求将之前的所有被 kill 掉的囚犯的编号输出。

输入

题中描述的囚犯数 n （即编号为 1 至 n ， n 不大于 1000）和间隔数 k （ k 大于等于 2，小于 n ）

输出

顺序输出被 kill 掉的囚犯的编号，中间以空格隔开

样例输入

10 2

样例输出

2 4 6 8 10 3 7 1 9

250204:多项式加法

描述

我们经常遇到两多项式相加的情况，在这里，我们就需要用程序来模拟实现把两个多项式相加到一起。首先，我们会有两个多项式，每个多项式是独立的一行，每个多项式由系数、幂数这样的多个整数对来表示。

如多项式 $2x^{20} - x^{17} + 5x^9 - 7x^7 + 16x^5 + 10x^4 + 22x^2 - 15$

对应的表达式为：2 20 -1 17 5 9 -7 7 16 5 10 4 22 2 -15 0。

为了标记每行多项式的结束，在表达式后面加上了一个幂数为负数的整数对。

同时输入表达式的幂数大小顺序是随机的。

我们需要做的就是将所给的两个多项式加起来。

输入

输入包括多行。

第一行整数 n ，表示有多少组的多项式需要求和。($1 < n < 100$)

下面为 $2n$ 行整数，每一行都是一个多项式的表达式。表示 n 组需要相加的多项式。每行长度小于 300。

输出

输出包括 n 行，每行为 1 组多项式相加的结果。

在每一行的输出结果中，多项式的每一项用 $[x\ y]$ 形式的字符串表示， x 是该项的系数、 y 是该项的幂数。要求按照每一项的幂从高到低排列，即先输出幂数高的项、再输出幂数低的项。

系数为零的项不要输出。

样例输出

```
[ 2 20 ][ 2 19 ][ 2 17 ][ 15 10 ][ 5 9 ][ 6 5 ][ 14 4 ][ 35 2 ][ -22 0 ]
```

```
[ 2 23 ][ 2 19 ][ 2 17 ][ 15 10 ][ 6 8 ][ 8 7 ][ -3 5 ][ 44 4 ][ 22 2 ][ -18 0 ]
```

提示

第一组样例数据的第二行末尾的 $8 -8$ ，因为幂次 -8 为负数，所以这一行数据结束， $8 -8$ 不要参与计算。

250205:放苹果（盘子相同）

65535ms

内存限制:

65535kB

描述

把 M 个同样的苹果放在 N 个同样的盘子里，允许有的盘子空着不放，问共有多少种不同的分法？（用 K 表示） $5, 1, 1$ 和 $1, 5, 1$ 是同一种分法。

输入

苹果个数 m 和盘子个数 n ($0 \leq m, 1 \leq n \leq 10$)

输出

不同的放法数目

样例输入

7 3

样例输出

8

250206:波兰表达式

描述

波兰表达式是一种把运算符前置的算术表达式，例如普通的表达式 $2 + 3$ 的波兰表示法为 $+ 2 3$ 。波兰表达式的优点是运算符之间不必有优先级关系，也不必用括号改变运算次序，例如 $(2 + 3) * 4$ 的波兰表示法为 $* + 2 3 4$ 。本题求解波兰表达式的值，其中运算符包括 $+ - * /$ 四个。

输入

输入为一行，其中运算符和运算数之间都用空格分隔，运算数是浮点数。

输出

输出为一行，表达式的值，保留一位小数。

样例输入

```
* + 11.0 12.0 + 24.0 35.0
```

样例输出

```
1357.0
```

250207:Queue

描述

队列的特点就是先进先出。对于一个元素 i 进队列，我们定义为 $+i$ ，对于一个元素 i 出队列，我们定义为 $-i$ ，由此我们可以把对队列的操作翻译为一个序列，我们定义为队列序列。对于"+1 -1 +2 +3 -2 +4"的操作，最后队列中保留两个元素 3 和 4。现在我们给出你一个个队列序列，请告诉我们这些是不是合法的队列序列。

输入

首先第一行为一个整数 T 代表样例的数目。

对于每个样例，第一行为整数 n ，代表序列中有 n 个操作。

第二行会有 n 个空格隔开的，格式为" $+i$ "或者" $-i$ "的操作，保证 i 不小于 0。

输出

对于第 i 个样例，

如果是合法的队列序列，输出

Case i: yes

否则输出

Case i: no

样例输入

1

8

+1 -1 +2 +3 -2 +4 -3 -4

样例输出

Case 1: yes

250208:位查询（数据加强版）

描述

本题是 5345 位查询的数据加强版，没有任何优化的做法会超时

描述

给出 N 个范围在 $[0, 65535]$ 的整数，编程支持以下的操作：

- (1) 修改操作：C d，所有的数都增加 d。如果超过 65535，把结果模 65536。 $0 \leq d \leq 65535$
- (2) 查询操作：Q i，统计在 N 个正整数中有多少个整数其对应的二进制形式的第 i 位二进制位为非 0。 $0 \leq i \leq 15$ 。并且最低位 i 为 0。

最后，输出所有查询操作的统计值。

输入

输入的第一行为两个正整数 N,M,其中 N 为操作的整数的个数，而 M 为具体有多少个操作。

输入的第二行为 N 个正整数，为进行操作的 N 个正整数。

下面有 M 行，分别表示 M 个操作。

$N \leq 10000, M \leq 10000$

输出

输出所有查询操作 Q 的统计值，每一个查询操作统计结果输出为一行。

提示

借助前缀和优化降低每次查询的复杂度

```
n,m=map(int,input().split())  
ans=[]  
alist=[i for i in range(1,n+1)]  
k=0  
while len(alist)>1:  
    k+=1  
    if k%m==0:  
        a=alist.pop(0)  
        ans.append(a)  
    else:  
        alist.append(alist.pop(0))  
print(" ".join(map(str,ans)))
```

```
from collections import deque
```

```
import sys
```

```
def main():
```

```
    T = int(sys.stdin.readline())
```

```

for case in range(1, T + 1):

    n = int(sys.stdin.readline().strip())

    ops = sys.stdin.readline().split()

    queue = deque()

    valid = True

    for op in ops:

        if op[0] not in '+-':

            valid = False

            break

        num = int(op[1:])

        if op[0] == '+':

            queue.append(num)

        else: # op[0] == '-'

            if not queue:

                valid = False

                break

            front = queue.popleft()

            if front != num:

                valid = False

                break

    print(f"Case {case}: {'yes' if valid else 'no'}")

```

```

if __name__ == "__main__":

```

```

    main()

```

```
N,M=map(int,input().split())
alist=list(map(int,input().split()))
for i in range(M):
    a=input()
    if a[0]=="C":
        d=int(a[2])
        for j in range(N):
            alist[j]=(alist[j]+d) % 65535
    if a[0]=="Q":
        k=int(a[2])
        count=0
        for l in range(N):
            if bin(alist[l])[-k-1]=="1":
                count+=1
        print(count)
```

250301:简单的整数划分问题

描述

将正整数 n 表示成一系列正整数之和, $n=n_1+n_2+\cdots+n_k$, 其中 $n_1\geq n_2\geq\cdots\geq n_k\geq 1$, $k\geq 1$ 。

正整数 n 的这种表示称为正整数 n 的划分。正整数 n 的不同的划分个数称为正整数 n 的划分数。

输入

标准的输入包含若干组测试数据。每组测试数据是一个整数 $N(0 < N \leq 50)$ 。

输出

对于每组测试数据, 输出 N 的划分数。

样例输入

5

样例输出

7

提示

5, 4+1, 3+2, 3+1+1, 2+2+1, 2+1+1+1, 1+1+1+1+1

250302:采药

描述

辰辰是个很有潜能、天资聪颖的孩子，他的梦想是称为世界上最伟大的医师。为此，他想拜附近最有威望的医师为师。医师为了判断他的资质，给他出了一个难题。医师把他带到个到处都是草药的山洞里对他说：“孩子，这个山洞里有一些不同的草药，采每一株都需要一些时间，每一株也有它自身的价值。我会给你一段时间，在这段时间里，你可以采到一些草药。如果你是一个聪明的孩子，你应该可以让采到的草药的总价值最大。”

如果你是辰辰，你能完成这个任务吗？

输入

输入的第一行有两个整数 T ($1 \leq T \leq 1000$) 和 M ($1 \leq M \leq 100$)， T 代表总共能够用来采药的时间， M 代表山洞里的草药的数目。接下来的 M 行每行包括两个在 1 到 100 之间（包括 1 和 100）的整数，分别表示采摘某株草药的时间和这株草药的价值。

输出

输出只包括一行，这一行只包含一个整数，表示在规定的时间内，可以采到的草药的最大总价值。

样例输入

```
70 3
71 100
69 1
1 2
```

样例输出

```
3
```

250303:拦截导弹

总时间限制: 1000ms 内存限制: 65536kB

描述

某国为了防御敌国的导弹袭击，发展出一种导弹拦截系统。但是这种导弹拦截系统有一个缺陷：虽然它的第一发炮弹能够到达任意的高度，但是以后每一发炮弹都不能高于前一发的高度（可以等于）。某天，雷达捕捉到敌国的导弹来袭。由于该系统还在试用阶段，所以只有一套系统，因此有可能不能拦截所有的导弹。

输入导弹依次飞来的高度（雷达给出的高度数据是不大于 30000 的正整数），计算这套系统最多能拦截多少导弹。

输入

第一行是一个整数 N ($1 \leq N \leq 15$)，表示导弹数。

第二行包含 N 个整数，为导弹依次飞来的高度（雷达给出的高度数据是不大于 30000 的正整数）。

输出

一个整数，表示最多能拦截的导弹数。

样例输入

8

389 207 155 300 299 170 158 65

样例输出

6

250305:小木棍

描述

小明将一批等长的木棍随机切成最长为 50 单位的小段。现在他想要将木棍还原成原来的状态，但是却忘记了原来的木棍数量和长度。请写一个程序帮助他计算如果还原成原来的等长木棍，其长度可能的最小值。所有的长度均大于 0。

输入

输入包含多个实例。每个实例有两行，第一行是切割后的木棍数量 n （最多 64 个），第二行为 n 个以空格分开的整数，分别为每根木棍的长度。输入的最后以 n 为 0 结束。

输出

对于每个实例，输出一行其长度的可能的最小值。

样例输入

```
9
5 2 1 5 2 1 5 2 1
4
1 2 3 4
0
```

样例输出

```
6
5
、
、、
```


250306:奇数拆分

描述

输入一个正整数 N 。把 N 拆成若干个(至少一个)两两不同的正奇数的和，请问有多少种拆分方法？

输入

一个正整数 N 。($1 \leq N \leq 100$)

输出

先按字典序输出所有的拆分方案，每个方案一行。

最后一行输出方案总数

样例输入

15

样例输出

1 3 11

1 5 9

3 5 7

15

4

250307:回文字符串

- [查看](#)
- [提交](#)
- [统计](#)
- [提问](#)

总时间限制:

10000ms

单个测试点时间限制:

1000ms

内存限制:

262144kB

描述

给定一个字符串 S ，最少需要几次增删改操作可以把 S 变成一个回文字符串？

一次操作可以在任意位置插入一个字符，或者删除任意一个字符，或者把任意一个字符修改成任意其他字符。

输入

字符串 S 。 S 的长度不超过 100，只包含 'A'-'Z'。

输出

最少的修改次数。

样例输入

ABAD

样例输出、

250308:最长的公共子序列的长度

总时间限制:

1000ms

内存限制:

10240kB

描述

我们称一个字符的数组 S 为一个序列。对于另外一个字符数组 Z ,如果满足以下条件,则称 Z 是 S 的一个子序列: (1) Z 中的每个元素都是 S 中的元素 (2) Z 中元素的顺序与在 S 中的顺序一致。例如: 当 $S = (E,R,C,D,F,A,K)$ 时, (E, C, F) 和 (E, R) 等等都是它的子序列。而 (R, E) 则不是。

现在我们给定两个序列, 求它们最长的公共子序列的长度。

输入

一共两行, 分别输入两个序列。

输出

一行, 输出最长公共子序列的长度。

样例输入

ABCB DAB

BDCABA

样例输出

4

250601:献给阿尔吉侬的花束

描述

阿尔吉侬是一只聪明又慵懒的小白鼠，它最擅长的就是走各种各样的迷宫。今天它要挑战一个非常大的迷宫，研究员们为了鼓励阿尔吉侬尽快到达终点，就在终点放了一块阿尔吉侬最喜欢的奶酪。现在研究员们想知道，如果阿尔吉侬足够聪明，它最少需要多少时间就能吃到奶酪。

迷宫用一个 $R \times C$ 的字符矩阵来表示。字符 S 表示阿尔吉侬所在的位置，字符 E 表示奶酪所在的位置，字符 # 表示墙壁，字符 . 表示可以通行。阿尔吉侬在 1 个单位时间内可以从当前的位置走到它上下左右四个方向上的任意一个位置，但不能走出地图边界。

输入

第一行是一个正整数 T ($1 \leq T \leq 10$)，表示一共有 T 组数据。

每一组数据的第一行包含了两个用空格分开的正整数 R 和 C ($2 \leq R, C \leq 200$)，表示地图是一个 $R \times C$ 的矩阵。

接下来的 R 行描述了地图的具体内容，每一行包含了 C 个字符。字符含义如题目描述中所述。保证有且仅有一个 S 和 E。

输出

对于每一组数据，输出阿尔吉侬吃到奶酪的最少单位时间。若阿尔吉侬无法吃到奶酪，则输出“oop!”（只输出引号里面的内容，不输出引号）。每组数据的输出结果占一行。

250602:城堡问题

描述

```

      1   2   3   4   5   6   7
#####
1 #   |   #   |   #   |   |   #
   #####- - -#####- - -#- - -#####- - -#
2 #   #   |   #   #   #   #   #
   #- - -#####- - -#####- - -#####- - -#
3 #   |   |   #   #   #   #   #
   #- - -#####- - -#####- - -#- - -#
4 #   #   |   |   |   |   #   #
   #####
```

(图 1)

= Wall
| = No wall
- = No wall

图 1 是一个城堡的地形图。请你编写一个程序，计算城堡一共有多少房间，最大的房间有多大。城堡被分割成 $m \times n$ ($m \leq 50$, $n \leq 50$) 个方块，每个方块可以有 0~4 面墙。

输入

程序从标准输入设备读入数据。第 1、2 行每行 1 个整数，分别是南北向、东西向的方块数。在接下来的输入行里，每个方块用一个数字 ($0 \leq p \leq 50$) 描述。用一个数字表示方块周围的墙，1 表示西墙，2 表示北墙，4 表示东墙，8 表示南墙。每个方块用代表其周围墙的数字之和表示。城堡的内墙被计算两次，方块 (1,1) 的南墙同时也是方块 (2,1) 的北墙。输入的数据保证城堡至少有两个房间。

输出

输出 2 行，每行一个数，表示城堡的房间数、城堡中最大房间所包括的方块数。结果显示在标准输出设备上。

250603:棋盘问题

描述

在一个给定形状的棋盘（形状可能是不规则的）上面摆放棋子，棋子没有区别。要求摆放时任意的两个棋子不能放在棋盘中的同一行或者同一列，请编程求解对于给定形状和大小的棋盘，摆放 k 个棋子的所有可行的摆放方案 C 。

输入

输入含有多组测试数据。

每组数据的第一行是两个正整数， n k ，用一个空格隔开，表示了将在一个 $n \times n$ 的矩阵内描述棋盘，以及摆放棋子的数目。 $n \leq 8, k \leq n$

当为 -1 -1 时表示输入结束。

随后的 n 行描述了棋盘的形状：每行有 n 个字符，其中 $\#$ 表示棋盘区域， $.$ 表示空白区域（数据保证不出现多余的空白行或者空白列）。

输出

对于每一组数据，给出一行输出，输出摆放的方案数目 C （数据保证 $C < 2^{31}$ ）。

样例输入

2 1

$\#.$

$. \#$

4 4

$... \#$

$.. \#.$

$. \#..$

$\#...$

-1 -1

样例输出

2

1

250604:拯救行动

描述

公主被恶人抓走，被关押在牢房的某个地方。牢房用 $N \times M$ ($N, M \leq 200$) 的矩阵来表示。矩阵中的每项可以代表道路 (@)、墙壁 (#)、和守卫 (x)。

英勇的骑士 (r) 决定孤身一人去拯救公主 (a)。我们假设拯救成功的表示是“骑士到达了公主所在的位置”。由于在通往公主所在位置的道路中可能遇到守卫，骑士一旦遇到守卫，必须杀死守卫才能继续前进。

现假设骑士可以向上、下、左、右四个方向移动，每移动一个位置需要 1 个单位时间，杀死一个守卫需要花费额外的 1 个单位时间。同时假设骑士足够强壮，有能力杀死所有的守卫。

给定牢房矩阵，公主、骑士和守卫在矩阵中的位置，请你计算拯救行动成功需要花费最短时间。

输入

第一行为一个整数 S ，表示输入的数据的组数（多组输入）

随后有 S 组数据，每组数据按如下格式输入

1、两个整数代表 N 和 M , ($N, M \leq 200$).

2、随后 N 行，每行有 M 个字符。“@”代表道路，“a”代表公主，“r”代表骑士，“x”代表守卫，“#”代表墙壁。

输出

如果拯救行动成功，输出一个整数，表示行动的最短时间。

如果不可能成功，输出 "Impossible"

样例输入

2

7 8

@ # # # # # @

@ a # @ @ r @

@ @ # x @ @ @

@ @ # @ @ # @ #

@ @ @ # # @ @

@#@#@#@#@

@#@#@#@#@

13 40

@x@@##x@#x@x#xxx##@#x@x@@#x#@#x#@#x@#@x

xx###x@x#@@##xx@@@#@x@@#x@xx@@#x@#x@@x@

#@x#@x#x#@@##@x#@xx#xxx@x##@@@#@x@@x@x

@##x@@@x#xx#@@#xxx#@@x@x@#x@@@x@#@#x@#

@#xxxxx##@x##x@xx@#@#x@x####@x#x##@#@

#xxx#@#x##xxx@#@xx@#@x@xx#@#xxx@x#####

#x@xxx#@x@@@@##@x#xx#xxx@#xx#@#####x#@x

xx##@#@x##x##x#@x#@a#xx@##@#@##xx@#@@x@x

x#x#@x@#x#@##@rx@x#xxx@##x##xx#@#x@xx@

#x@@@@###x##x@x#@@#@@x@x@xx@@@@@##@x@x@x

x#xx@x###@xxx#@#x#@@###@#@##@x#@x@#@#@@

#@#x@x#x#x###@x@@@@xxx####x@x##@x####xx#@x

#x#@x#x#####@#@#x@#xxx#xx@@@@#xx#x#####@

样例输出

13

7

\\

250605：整理一切

描述

在非重复值的升序排序序列中，使用某种形式的小于运算符按从小到大的顺序对元素进行排序。例如，排序序列 A、B、C、D 意味着 $A < B$ 、 $B < C$ 和 $C < D$ 。在这个问题中，我们将为您提供一组 $A < B$ 形式的关系，并要求您确定是否已指定排序顺序。

输入

输入由多个问题实例组成。每个实例都以包含两个正整数 n 和 m 的行开头。第一个值表示要排序的对象数，其中 $2 \leq n \leq 26$ 。要排序的对象将是大写字母表的前 n 个字符。第二个值 m 表示 $A < B$ 形式的关系数量，这将在这个问题实例中给出。接下来是 m 行，每行都包含一个由三个字符组成的此类关系：一个大写字母、字符 “<” 和第二个大写字母。没有字母会超出字母表的前 n 个字母的范围。值 $n = m = 0$ 表示输入结束。

输出

对于每个问题实例，output 由一行组成。这行应该是以下三个之一：

xxx relations: yyy...y.

无法确定排序顺序。

在 xxx 关系后发现不一致。

其中 xxx 是确定排序序列或发现不一致时处理的关系数，以先到者为准，yyy...y 是排序的升序序列。

样例输入

4 6

A<B

A<C

B<C

C<D

B<D

A<B

3 2

A<B

B<A

26 1

A<Z

0 0

样例输出

Sorted sequence determined after 4 relations: ABCD.

Inconsistency found after 2 relations.

Sorted sequence cannot be determined.

250606：地铁

描述

您刚刚从一个安静的滑铁卢社区搬到一个嘈杂的大城市。您现在不必每天骑自行车上学，而是可以步行和乘坐地铁。因为你不想上课迟到，所以你想知道你需要多长时间才能到学校。

您以 10 公里/小时的速度行走。地铁以 40 公里/小时的速度行驶。假设你很幸运，每当您到达地铁站时，都有一列火车，您可以立即登上。您可以多次上下地铁，如果您愿意，您可以在不同的地铁线路之间切换。所有地铁线路都是双向的。

输入

输入包括您家和学校的 x, y 坐标，后跟多条地铁线路的规格。每条地铁线路由线路上每个站点的非负整数 x, y 坐标按顺序组成。您可以假设地铁在相邻站点之间沿直线运行，坐标代表整数米。每条线路至少有两个停靠点。每条地铁线的终点后跟虚拟坐标对 $-1, -1$ 。该市总共最多有 200 个地铁站。

输出

输出是您到达学校所需的分钟数，四舍五入到最接近的分钟，采用最快的路线。

样例输入

```
0 0 10000 1000
```

```
0 200 5000 200 7000 200 -1 -1
```

```
2000 600 5000 600 10000 600 -1 -1
```

样例输出

```
21
```

250607:兔子与樱花

描述

很久很久之前，森林里住着一群兔子。有一天，兔子们希望去赏樱花，但当他们到了上野公园门口却忘记了带地图。现在兔子们想求助于你来帮他们找到公园里的最短路。

输入

输入分为三个部分。

第一个部分有 $P+1$ 行 ($P < 30$)，第一行为一个整数 P ，之后的 P 行表示上野公园的地点，字符串长度不超过 20。

第二个部分有 $Q+1$ 行 ($Q < 50$)，第一行为一个整数 Q ，之后的 Q 行每行分别为两个字符串与一个整数，表示这两点有直线的道路，并显示二者之间的距离（单位为米）。

第三个部分有 $R+1$ 行 ($R < 20$)，第一行为一个整数 R ，之后的 R 行每行为两个字符串，表示要求的路线。

输出

输出有 R 行，分别表示每个路线最短的走法。其中两个点之间，用 $\rightarrow$ (距离) $\rightarrow$ 相隔。

样例输入

6

Ginza

Sensouji

Shinjukugyoen

Uenokouen

Yoyogikouen

Meijishinguu

6

Ginza Sensouji 80

Shinjukugyoen Sensouji 40

Ginza Uenokouen 35

Uenokouen Shinjukugyoen 85

Sensouji Meijishinguu 60

Meijishinguu Yoyogikouen 35

2

Uenokouen Yoyogikouen

Meijishinguu Meijishinguu

样例输出

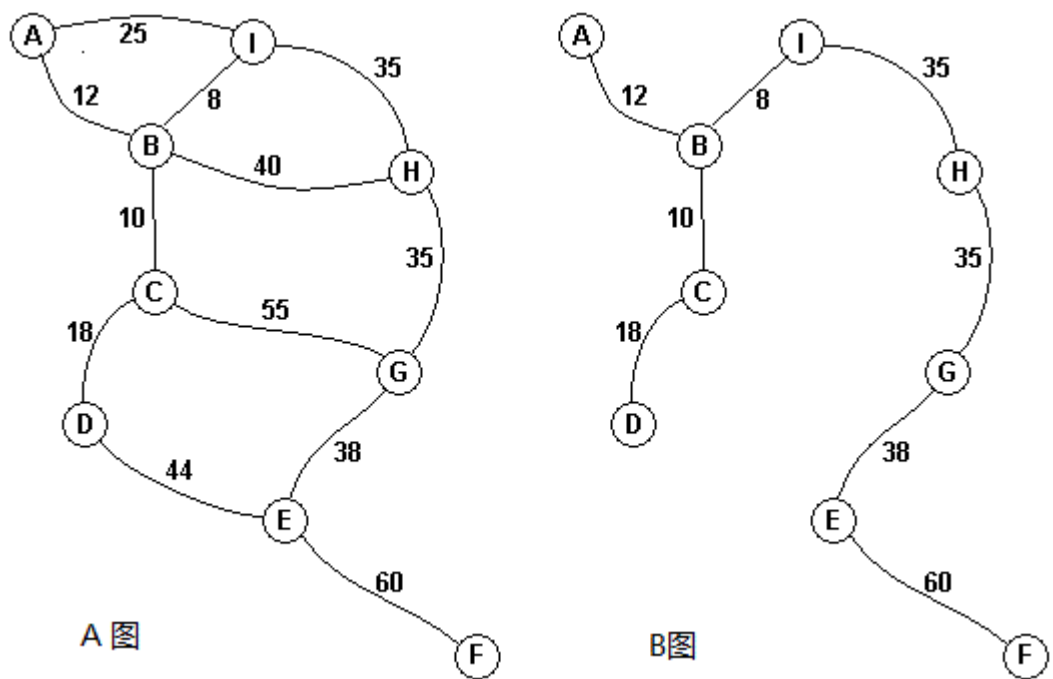
Uenokouen->(35)->Ginza->(80)->Sensouji->(60)->Meijishinguu->(35)->Yoyogikoue
n

Meijishinguu

250608:兔子与星空

描述

很久很久以前，森林里住着一群兔子。兔子们无聊的时候就喜欢研究星座。如图所示，天空中已经有了 n 颗星星，其中有些星星有边相连。兔子们希望删除掉一些边，然后使得保留下的边仍能为 n 颗星星连通。他们希望计算，保留的边的权值之和最小是多少？



输入

第一行只包含一个表示星星个数的数 n ， n 不大于 26，并且这 n 个星星是由大写字母表里的前 n 个字母表示。接下来的 $n-1$ 行是由字母表的前 $n-1$ 个字母开头。最后一个星星表示的字母不用输入。对于每一行，以每个星星表示的字母开头，然后后面跟着一个数字，表示有多少条边可以从这个星星到后面字母表中的星星。如果 k 是大于 0，表示该行后面会表示 k 条边的 k 个数据。每条边的数据是由表示连接到另一端星星的字母和该边的权值组成。权值是正整数的并且小于 100。该行的所有数据字段分隔单一空白。该星星网络将始终连接所有的星星。该星星网络将永远不会超过 75 条边。没有任何一个星星会有超过 15 条的边连接到其他星星（之前或之后的字母）。在下面的示例输入，数据是与上面的图相一致的。

输出

输出是一个整数，表示最小的权值和

样例输入

9

A 2 B 12 I 25

B 3 C 10 H 40 I 8

C 2 D 18 G 55

D 1 E 44

E 2 F 60 G 38

F 0

G 1 H 35

H 1 I 35

样例输出

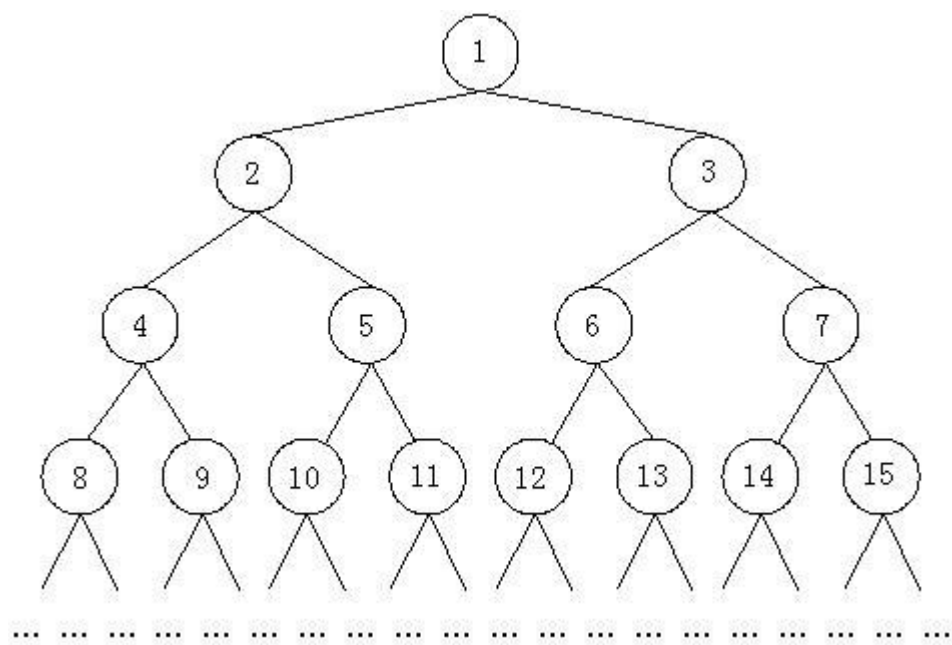
216

提示

考虑看成最小生成树问题，注意输入表示。

1:二叉树

描述



如上图所示，由正整数 1, 2, 3……组成了一颗二叉树。我们已知这个二叉树的最后一个结点是 n 。现在的问题是，结点 m 所在的子树中一共包括多少个结点。

比如， $n = 12$ ， $m = 3$ 那么上图中的结点 13, 14, 15 以及后面的结点都是不存在的，结点 m 所在子树中包括的结点有 3, 6, 7, 12，因此结点 m 的所在子树中共有 4 个结点。

输入

输入数据包括多行，每行给出一组测试数据，包括两个整数 m , n ($1 \leq m \leq n \leq 1000000000$)。最后一组测试数据中包括两个 0，表示输入的结束，这组数据不用处理。

输出

对于每一组测试数据，输出一行，该行包含一个整数，给出结点 m 所在子树中包括的结点的数目。

样例输入

3 12

0 0

样例输出

4

```
import sys

for line in sys.stdin:

    line = line.strip()

    if not line:

        continue

    m,n=map(int,line.split())

    if m==0 and n==0:

        break

    else:

        levelnode=1

        count=0

        start=m

        while start<=n:

            end=start+levelnode-1

            if end<=n:

                count+=levelnode

            else:

                count+=n-start+1

            levelnode*=2

            start*=2

        print(count)
```

2:重建二叉树

描述

给定一棵二叉树的前序遍历和中序遍历的结果，求其后序遍历。

输入

输入可能有多组，以 EOF 结束。

每组输入包含两个字符串，分别为树的前序遍历和中序遍历。每个字符串中只包含大写字母且互不重复。

输出

对于每组输入，用一行来输出它后序遍历结果。

样例输入

DBACEGF ABCDEFG

BCAD CBAD

样例输出

ACBFGED

CDAB

```
def rebuild(s1,s2):
```

```
    if not s1:
```

```
        return ""
```

```
    if len(s1)==1:
```

```
        return s1
```

```
    root=s1[0]
```

```
s3,s4,s5,s6="", "", "", ""
```

```
for i in range(len(s2)):
```

```
    if s2[i]==root:
```

```
        s3,s4=s1[1:i+1],s1[i+1:]
```

```
        s5,s6=s2[:i],s2[i+1:]
```

```
        break
```

```
    return rebuild(s3,s5)+rebuild(s4,s6)+root
```

```
import sys
```

```
for line in sys.stdin:
```

```
    line = line.strip()
```

```
    if line=="EOF":
```

```
        break
```

```
    s1,s2=map(str,line.split())
```

```
    print(rebuild(s1,s2))
```

3:二叉搜索树

查看提交统计提问

总时间限制: 1000ms 内存限制: 1024kB

描述

二叉搜索树在动态查表中有特别的用处，一个无序序列可以通过构造一棵二叉搜索树变成一个有序序列，构造树的过程即为对无序序列进行排序的过程。每次插入的新的结点都是二叉搜索树上新的叶子结点，在进行插入操作时，不必移动其它结点，只需改动某个结点的指针，由空变为非空即可。

这里，我们想探究二叉树的建立和序列输出。

输入

只有一行，包含若干个数字，中间用空格隔开。(数字可能会有重复)

输出

输出一行，对输入数字建立二叉搜索树后进行前序周游的结果。

样例输入

41 467 334 500 169 724 478 358 962 464 705 145 281 827 961 491 995 942 827 436

样例输出

41 467 334 169 145 281 358 464 436 500 478 491 724 705 962 827 961 942 995

```
class TreeNode:
```

```
    def __init__(self, value):
```

```
        self.value = value
```

```
        self.left = None
```

```

        self.right = None

def insert_node(root, value):

    if root == None:

        return TreeNode(value)

    if value < root.value:

        root.left = insert_node(root.left, value)

    elif value > root.value:

        root.right = insert_node(root.right, value)

    return root

def preorder(root):

    result = []

    def t(node):

        if node:

            result.append(str(node.value))

            t(node.left)

            t(node.right)

    t(root)

    return " ".join(result)

s=input().strip()

if s:

    lst=list(map(int,s.split()))

    root=None

    for i in lst:

        root = insert_node(root, i)

    print(preorder(root))

else:

    print()

```

4:树的转换

描述

我们都知道用“左儿子右兄弟”的方法可以将一棵一般的树转换为二叉树，如：

现在请你将一些一般的树用这种方法转换为二叉树，并输出转换前和转换后树的高度。

输入

输入是一个由“u”和“d”组成的字符串，表示一棵树的深度优先搜索信息。比如，dudduduudu 可以用来表示上文中的左树，因为搜索过程为：0 Down to 1 Up to 0 Down to 2 Down to 4 Up to 2 Down to 5 Up to 2 Up to 0 Down to 3 Up to 0。

你可以认为每棵树的结点数至少为 2，并且不超过 10000。

输出

按如下格式输出转换前和转换后树的高度：

h1 => h2

其中，h1 是转换前树的高度，h2 是转换后树的高度。

样例输入

dudduduudu

样例输出

2 => 4

```
class TreeNode:
```

```
    def __init__(self):
```

```
        self.parent=None
```

```
        self.children=[]
```

```
        self.next_sibling=None
```

```
        self.left = None
```

```

        self.right = None

        self.depth=0

s=input().strip()

root=TreeNode()

nodes=[root]

stack=[root]

for c in s:

    if c=="d":

        parent=stack[-1]

        newnode=TreeNode()

        newnode.parent=parent

        newnode.depth=parent.depth+1

        parent.children.append(newnode)

        if len(parent.children)>=2:

            prev_sibling=parent.children[-2]

            prev_sibling.next_sibling=newnode

            newnode.next_sibling=None

            stack.append(newnode)

            nodes.append(newnode)

        elif c=="u":

            stack.pop()

for node in nodes:

    if node.children:

        node.left=node.children[0]

    else:

        node.left=None

    node.right=node.next_sibling

```

```

h1=max(node.depth for node in nodes)

max_depth={}

post_order=[]

current=root

while True:

    while current:

        post_order.append((current,False))

        current=current.left

    if not post_order:

        break

    node,processed=post_order.pop()

    if not processed:

        post_order.append((node,True))

        current=node.right

    else:

        leftdepth=max_depth.get(node.left,0)

        rightdepth=max_depth.get(node.right,0)

        currentmax=max(leftdepth,rightdepth)+1

        max_depth[node]=currentmax

h2 = max_depth.get(root, 0) - 1

print(f"{h1} => {h2}")

```


5:合并果子

描述

有 n 堆果子 ($n \leq 10000$)，多多决定把所有的果子合成一堆。

每一次合并，多多可以把两堆果子合并到一起，消耗的体力等于两堆果子数量之和。可以看出，所有的果子经过 $n-1$ 次合并之后，就只剩下一堆了。多多在合并果子时总共消耗的体力等于每次合并所耗体力之和。

设计出合并的次序方案，使多多耗费的体力最少，并输出这个最小的体力耗费值。

输入

两行，第一行是一个整数 n ($1 \leq n \leq 10000$)，表示果子的种类数。
第二行包含 n 个整数，用空格分隔，第 i 个整数 a_i ($1 \leq a_i \leq 20000$) 是第 i 堆果子的数目。

输出

一行，这一行只包含一个整数，也就是最小的体力耗费值。输入数据保证这个值小于 2^{31} 。

样例输入

3

1 2 9

样例输出

15

#优先队列，最小堆

```
import heapq

n = int(input())

if n==0 or n==1:
    print(0)
else:
    a=list(map(int,input().split()))
    total=0
    heapq.heapify(a)
    while len(a)>1:
        x=heapq.heappop(a)
        y=heapq.heappop(a)
        s=x+y
        total+=s
        heapq.heappush(a,s)
    print(total)
```

6:树的重量

描述

有一棵 k 层的满二叉树（一共有 2^k-1 个节点，且从上到下从左到右依次编号为 $1, 2, \dots, 2^k-1$ ），最开始每个节点的重量的均为 0 。请编程实现如下两种操作：

- $1\ x\ y$: 给以 x 为根的子树的每个节点的重量的分别增加 y （ y 是整数且绝对值不超过 100 ）
- $2\ x$: 查询（此时的）以 x 为根的子树的所有节点重量之和

输入

输入有 $n+1$ 行。第一行是两个整数 k, n ，分别表示满二叉树的层数和操作的个数。接下来 n 行，每行形如 $1\ x\ y$ 或 $2\ x$ ，表示一个操作。

$k \leq 15$ （即最多 32767 个节点）， $n \leq 50000$ 。

输出

输出有若干行，对每个查询操作依次输出结果，每个结果占一行。

样例输入

```
3 7
1 2 1
2 4
1 6 3
2 1
1 3 -2
1 4 1
2 3
```

样例输出

```
1
6
```

-3

提示

可以通过对数计算某节点的深度：

```
import math
```

```
math.log2(x) #以小数形式返回 x 的对数值，注意 x 不能为 0
```

```
import sys
```

```
import math
```

```
def main():
```

```
    sys.setrecursionlimit(1 << 25)
```

```
    k, n = map(int, sys.stdin.readline().split())
```

```
    max_node = (1 << k) - 1
```

```
    L = [0] * (max_node + 1)
```

```
    R = [0] * (max_node + 1)
```

```
    size = [0] * (max_node + 1)
```

```
    current_pre = [1] # 使用列表来传递可变值
```

```
    def dfs(x):
```

```
        L[x] = current_pre[0]
```

```
        current_pre[0] += 1
```

```
        s = 1
```

```

left = 2 * x
right = 2 * x + 1
if left <= max_node:
    dfs(left)
    s += size[left]
if right <= max_node:
    dfs(right)
    s += size[right]
size[x] = s
R[x] = L[x] + s - 1

```

```
dfs(1)
```

```

class BIT:
    def __init__(self, n_size):
        self.n = n_size
        self.tree = [0] * (self.n + 2)

    def update(self, idx, delta):
        while idx <= self.n:
            self.tree[idx] += delta
            idx += idx & -idx

    def query(self, idx):
        res = 0
        while idx > 0:
            res += self.tree[idx]

```

```
        idx -= idx & -idx
    return res
```

```
bit = BIT(max_node)
sum_ops = [0] * (max_node + 1)
```

```
for _ in range(n):
    parts = sys.stdin.readline().split()
    if parts[0] == '1':
        x = int(parts[1])
        y = int(parts[2])
        sum_ops[x] += y
        bit.update(L[x], y * size[x])
    else:
        x = int(parts[1])
        current = x
        sum_ancestors = 0
        while current >= 1:
            sum_ancestors += sum_ops[current]
            current = current // 2
        part1 = sum_ancestors * size[x]
        part2 = bit.query(R[x]) - bit.query(L[x] - 1)
        part2 -= sum_ops[x] * size[x]
        print(part1 + part2)
```

```
if __name__ == '__main__':
    main()
```

7:简单的树上最近公共祖先查询

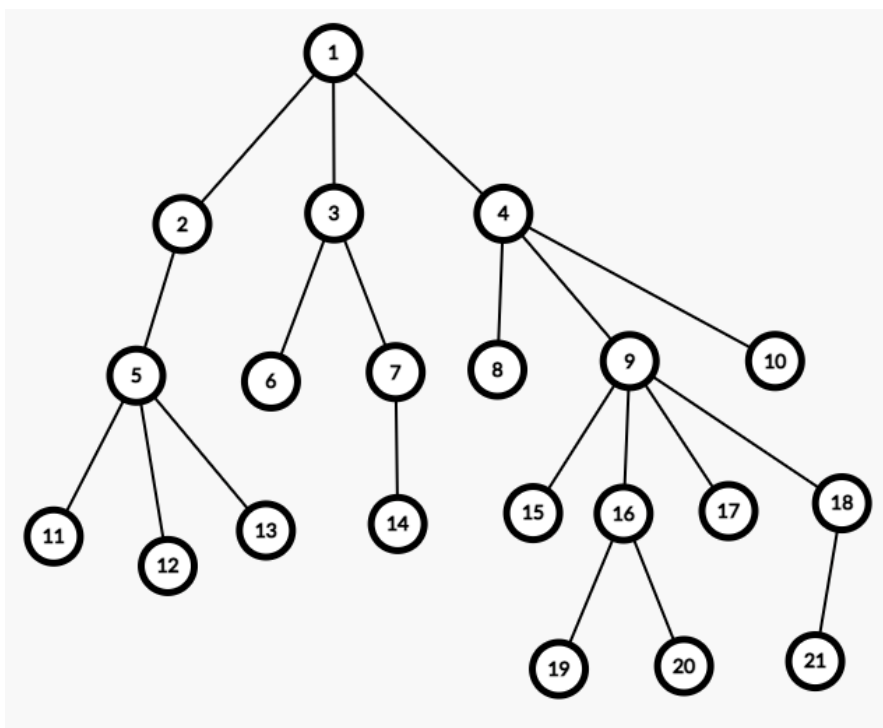
描述

给出一棵包含 N 个节点的有根树，节点编号 1 到 N ，根节点为 R 。请回答 Q 个查询，计算节点 u 和节点 v 的最近公共祖先。

祖先定义：任意节点 u 到根节点 R 的路径上的所有点（包括 u 和 R 在内）称为节点 u 的祖先。

最近公共祖先定义：节点 u 和节点 v 的公共祖先中，深度最深的那个节点被称为节点 u 和 v 的最近公共祖先（也可以直观理解为距离它们最近的公共祖先）。

例如，下图是以 1 号节点为根的有根树，其中 11 和 13 的最近公共祖先为 5，2 和 8 的最近公共祖先为 1，19 和 21 的最近公共祖先为 9，16 和 1 的最近公共祖先为 1，15 和 15 的最近公共祖先为 15。



输入

第一行包含两个整数 N, R ($1 \leq R \leq N \leq 1000$) 分别表示节点总数和根结点编号。

接下来 $N-1$ 行，每行表示一条边，包含两个整数 u, v ($1 \leq u, v \leq N, u \neq v$)。输入保证全部节点构成一棵树。

接下来一行，包含一个整数 Q ($1 \leq Q \leq 2000$) 表示询问次数。

接下来 Q 行，每行两个整数 x, y ($1 \leq x, y \leq N$) 表示询问节点 x 和 节点 y 的最近公共祖先。

输出

一共 Q 行，每行一个整数表示答案。

```
from collections import deque

N,R=map(int,input().split())

adj=[[]for _ in range(N+1)]

for i in range(N-1):

    u,v=map(int,input().split())

    adj[u].append(v)

    adj[v].append(u)

parent=[0]*(N+1)

depth=[0]*(N+1)

visited=[False]*(N+1)

q=deque()

q.append(R)

visited[R]=True

parent[R]=0

while q:

    u=q.popleft()

    for v in adj[u]:

        if not visited[v] and v!=parent[u]:
```



```
parent[v]=u
depth[v]=depth[u]+1
visited[v]=True
q.append(v)
```

```
Q=int(input())
```

```
for i in range(Q):
```

```
    x,y=map(int,input().split())
```

```
    a,b=x,y
```

```
    while depth[a]>depth[b]:
```

```
        a=parent[a]
```

```
    while depth[b]>depth[a]:#把 a,b 调整到同一深度
```

```
        b=parent[b]
```

```
    while a!=b:
```

```
        a=parent[a]
```

```
        b=parent[b]
```

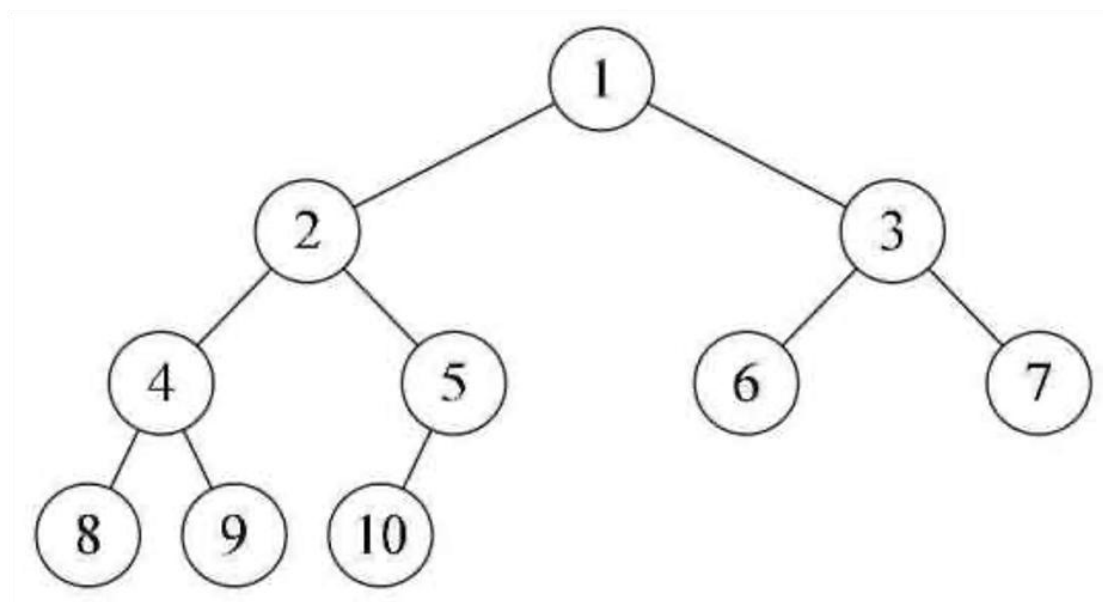
```
    print(a)
```

8:宝藏二叉树

描述

探险家小 B 发现了一颗宝藏二叉树。这棵树的树根为 Root，除了 Root 节点之外，每个节点均只有一个父节点，因此形成了一颗二叉树。宝藏二叉树的每个节点都有宝藏，每个宝藏具有相应的价值。小 B 希望摘取这些宝藏，使自己的收益最大。可是，宝藏二叉树有一个奇怪的性质，在摘取宝藏的时候，如果两个节点之间有边，那么最多只能摘取其中一个节点上的宝藏，如果因为贪婪而把两个节点上的宝藏都摘取，二叉树就会立即消失，丧失所有奖励。为此，小 B 求助于你，希望你能给出，小 B 在不使宝藏二叉树消失的前提下，能够获得宝藏的最大价值。

为了简化题目，规定宝藏二叉树均为完全二叉树，树中节点如图所示自上而下，自左向右，从 1-N 编号。



输入

输入分为两行

第一行为一个整数 N，代表二叉树中节点的个数。

第二行为一个 N 个非负整数。第 i 个数代表二叉树中编号为 i 的节点上的宝藏价值。

输出

输出为一个整数，代表小 B 的最大收益。

样例输入

6

3 4 5 1 3 1

样例输出

9

```
n = int(input())
values = list(map(int, input().split()))

selected = [0] * (n + 1)
unselected = [0] * (n + 1)

for i in range(n, 0, -1):
    left = 2 * i
    right = 2 * i + 1

    # 计算选中当前节点的最大值
    selected_val = values[i-1]
    if left <= n:
        selected_val += unselected[left]
    if right <= n:
        selected_val += unselected[right]
    selected[i] = selected_val

    # 计算不选中当前节点的最大值
```

```
unselected_val = 0

if left <= n:
    unselected_val += max(selected[left], unselected[left])

if right <= n:
    unselected_val += max(selected[right], unselected[right])

unselected[i] = unselected_val

print(max(selected[1], unselected[1]))
```